

Exercise 20 — Understanding FastAPI Code Patterns

Exercise Description

Understand the advanced FastAPI sample's patterns (Part 1), trace the execution flow of `GET /admin/users` (Part 2), simplify the advanced concepts (Part 3), and extend the codebase with a new feature — an action log — using the same patterns (Part 4).

Files: - `app.py` — a runnable, in-memory adaptation preserving the sample's patterns + the new logging feature. - `test_app.py` — 6 tests, all passing.

The original sample used SQLAlchemy async. To make it runnable without external services, I adapted the persistence to an in-memory store while keeping the architecture (generic repository, DI layers, middleware, RBAC, lifespan) identical.

Part 1 — Patterns in the code

- **Generic Repository (`Repository[T]` , `UserRepository`):** a reusable data-access layer parameterised by entity type. `Generic[T]` lets one class serve users, logs, etc., with type safety — add a new entity by subclassing, not by copy-pasting CRUD.
- **Layered Dependency Injection:** `get_db` → provides the store; `get_current_user` depends on `oauth2_scheme` **and** `get_db`; `require_role("admin")` depends on `get_current_user`. Each endpoint declares only what it needs and FastAPI resolves the chain.
- **Service layer (`UserService` , `ActionLogService`):** business logic sits between endpoints and repositories, keeping path operations thin.
- **Middleware (`timing_middleware`):** cross-cutting concern (timing) applied to every request, adding an `X-Process-Time` header.
- **Role-Based Access Control:** `require_role` gate. *I converted the sample's decorator into a dependency factory*** — the idiomatic FastAPI form (composable, testable, and shown in OpenAPI).
- **Lifespan handler:** `@asynccontextmanager` seeds users on startup and would clean up on shutdown.

Part 2 — Execution flow of `GET /admin/users`

```
Request: GET /admin/users (Authorization: Bearer <jwt>)
1. timing_middleware starts a timer, calls the next handler
2. FastAPI resolves dependencies for the route:
    require_role("admin")
    └─ get_current_user
```

```

└─ oauth2_scheme → extracts the bearer token
└─ get_db         → provides the Database
  → decode+verify JWT → load user by username
└─ checker: is_superuser? no → HTTPException 403 (stop)
                        yes → return the User
3. Path operation runs: ActionLogService(db).record(user, "list_users")
4. returns db.users.list(skip, limit) as [UserOut]
5. timing_middleware stops the timer, sets X-Process-Time header
Response: 200 + list of users (or 401 if token invalid, 403 if not admin)

```

Part 3 — Simplifying the advanced concepts

- **asynccontextmanager / lifespan:** "run this setup once when the app starts, and this teardown once when it stops." Everything before `yield` is startup; after `yield` is shutdown. Solves "seed/connect on boot, clean up on exit" without global hacks.
- **TimingMiddleware:** "wrap every request: note the time, let it run, then stamp how long it took onto the response." A simple example is the `X-Process-Time` header here.
- **JWT auth flow (plain words):** log in with username/password → server returns a signed token → you send that token on later requests → the server verifies the signature and expiry to know who you are, without storing a session.

Part 4 — New feature: action logging (implemented with the same patterns)

- **ActionLog model** + a `Repository[ActionLog]` on the `Database` (reusing the generic repository).
- **ActionLogService** with `record(username, action)` and `all()` (mirrors `UserService`).
- **Wired in** at the two required points: `login` records `"login"`; `list_users` records `"list_users"`.
- **GET /admin/logs** (admin-only, via the same `require_role("admin")` dependency) returns the log.

Test run:

```
6 passed in 1.98s
```

Covers: timing header present, login + `/users/me`, non-admin 403, admin list, **the action log recording both a login and an admin action**, and unauthenticated 401s.

Reflection Questions

1. **How did implementing the feature help me understand the architecture?**
Adding the log took only a model + repository + service + two call sites — proving the layering makes new features additive, not invasive.

2. **Which patterns were most useful?** The generic `Repository[T]` (the log needed zero new data-access code) and dependency injection (`require_role` reused verbatim to protect `/admin/logs`).
 3. **How would I explain repository + DI to a colleague?** The repository is "the only thing that touches storage"; DI is "the framework hands each endpoint the exact collaborators it declares, resolving a chain of them for you."
 4. **How did tracing the flow help?** It showed exactly where the log calls belonged (in the path operation, after auth succeeds) and why RBAC must run before the handler.
-

Submission for Exercise 20 — Understanding FastAPI Code Patterns

1. **Explanation of the design patterns:** generic `Repository (Generic[T])`, layered dependency injection, a service layer, timing middleware, RBAC (as a dependency factory), and a lifespan handler — all in `app.py` and described above.
2. **Execution-flow trace for `/admin/users`** : the step-by-step sequence (middleware → `require_role` → `get_current_user` → `oauth2_scheme/get_db` → JWT verify → superuser check → handler → middleware header), including the 401/403 branches.
3. **Plain-language translation guide:** lifespan/`asyncontextmanager`, `TimingMiddleware`, and the JWT flow explained simply (Part 3).
4. **Logging feature + reflection:** an `ActionLog` model, generic-repository-backed `ActionLogService`, log calls on login and admin actions, and an admin-only `/admin/logs` endpoint — verified by `test_app.py` (**6 tests passing**); reflection answers above.